

Module 01: Object-Oriented Programming

Language: Java

Type: Unguided lab

Mode: individual practical work

Learning outcomes

By completing this module, you should be able to:

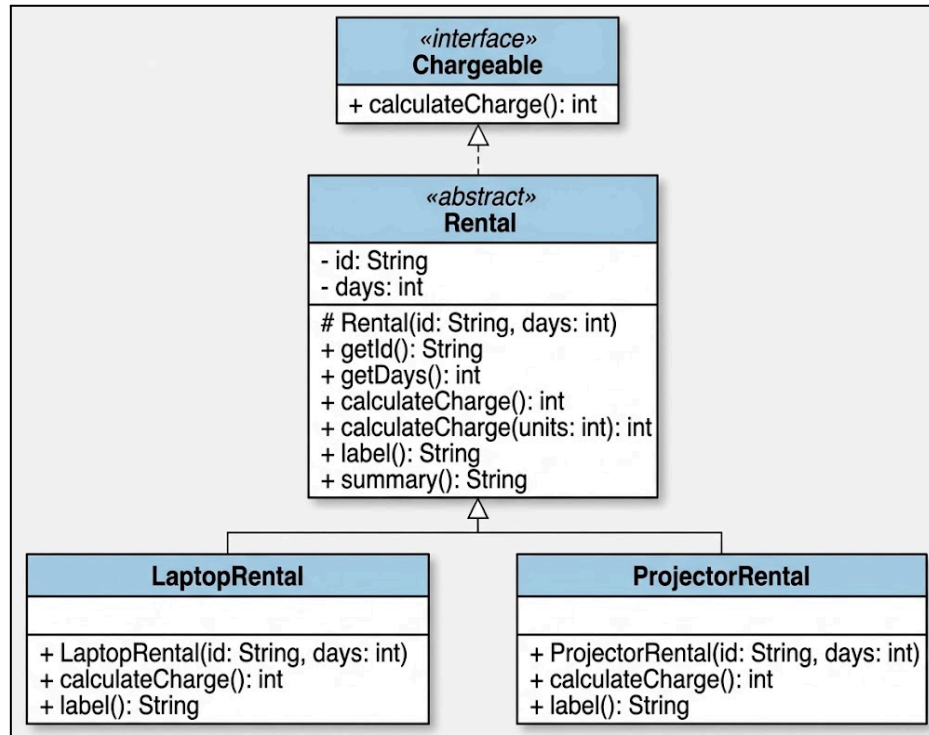
1. Distinguish a class, an object, and a reference variable.
2. Encapsulate state and initialise objects through constructors.
3. Implement inheritance, abstract classes, and an interface.
4. Distinguish method overloading from method overriding.
5. Explain and demonstrate runtime polymorphism through parent and interface references.
6. Test behaviour and justify a design using concrete examples from your code.

Concept Toolkit

Concept	Meaning and a useful distinction
Class and object	A class defines state and behaviour. Each object is an instance with its own identity and instance fields. A reference variable refers to an object; it is not the object itself.
Encapsulation	Keep fields private and expose appropriate operations. A getter can allow reading without allowing arbitrary external modification.
Inheritance	extends expresses an “is-a” relationship and allows a subclass to reuse accessible parent behaviour. Constructors are not inherited.
Overloading	Methods have the same name but different parameter lists. A different return type alone is insufficient. Overload selection uses compile-time types.
Overriding	A subclass supplies a new implementation of an inherited instance method with a matching signature and compatible return type. Use <code>@Override</code> .
Polymorphism	A parent or interface reference can refer to a subtype object. For an overridden instance method, the object’s runtime class determines the implementation executed.
Abstract class	Cannot be instantiated directly. May contain fields, constructors, concrete methods, and abstract methods. A concrete subclass must fulfil remaining abstract obligations.
Interface	Specifies a contract that implementing classes must fulfil. In this lab, its method is implicitly public abstract. Java interfaces can also have other kinds of methods; they are not used here.

Assessed Lab Case: Campus equipment rental

A campus media centre rents laptops and projectors. Each rental has an identifier and a positive number of days. Different equipment categories calculate charges differently, but a client should request a charge through the same contract. All amounts below are integer rupiah (IDR).



Exact Behaviour

- Rental implements Chargeable; both concrete classes extend Rental. Initialise inherited state using `super(...)`. No setters are required.
- LaptopRental: IDR 40,000 per day plus IDR 10,000 setup for each unit.
- ProjectorRental: first 3 days at IDR 60,000 per day; every day beyond 3 at IDR 45,000; add IDR 20,000 setup for each unit.
- `calculateCharge(int units)` returns `units * calculateCharge()`. Implement this overload once in Rental; do not duplicate it in the subclasses. Each unit uses the same duration; the setup fee is charged once per unit, not once per day.
- Parent `label()` returns "Rental"; subclass labels are "Laptop" and "Projector".
- `summary()` returns `id + " | " + label() + " | " + calculateCharge()`. Do not override `summary()`.
- Throw `IllegalArgumentException` if days or units are zero or negative. Assume valid non-empty IDs, days at most 30, and units at most 10. No additional upper-bound or ID validation is required.
- Input File (rentals.txt):
 - Input is read from a text file named `rentals.txt`.
 - The first line contains a positive integer representing the **total number of rental records**.
 - Each subsequent line represents one rental with 4 space-separated values: `<TYPE> <ID> <DAYS> <UNITS>`.
 - `<TYPE>` can be `LAPTOP` or `PROJECTOR`.
 - `<ID>` is the rental identifier (e.g., `R01`).

Labwork 01

- **<DAYS>** is a positive integer representing the number of rental days.
- **<UNITS>** is a positive integer representing the number of units rented.

Unguided Lab Tasks

B1. Implementation and demonstration

1. Implement the Chargeable interface and all three classes (Rental, LaptopRental, and ProjectorRental). Add @Override to each subclass override and to the abstract declaration implementing the interface method.
2. In Main, read rentals.txt using Scanner (scanner.next(), scanner.nextInt()).
3. Map each line based on its type into a concrete LaptopRental or ProjectorRental object.
4. Store the created objects in a **Rental[]** array.
5. Print each rental's summary() using a single loop through the Rental references. Do not select behaviour with instanceof, casts, or type-name conditionals (demonstrating runtime polymorphism).

B2. Required checks

Input file rentals.txt:

```
4
LAPTOP R01 2 1
PROJECTOR R02 5 2
PROJECTOR R03 3 2
PROJECTOR R04 4 1
```

The output of Main must produce the following summaries:

Rental / Input	Rate Calculation	Expected summary ()
LAPTOP R01 2 1	$((2 \times 40,000) + 10,000) \times 1$	R01 Laptop 90000
PROJECTOR R02 5 2	$((3 \times 60,000) + (2 \times 45,000) + 20,000) \times 2$	R02 Projector 580000
PROJECTOR R03 3 2	$((3 \times 60,000) + 20,000) \times 2$	R03 Projector 400000
PROJECTOR R04 4 1	$((3 \times 60,000) + (1 \times 45,000) + 20,000) \times 1$	R04 Projector 245000

The program execution must **display** exactly:

```
R01 | Laptop | 90000
R02 | Projector | 580000
R03 | Projector | 400000
R04 | Projector | 245000
```

Labwork 01

Submission

Please follow these instructions carefully to submit your work:

1. Create the `lw01` folder under your `dsa-[nrp]` repository, the repository you created previously in Module 00: Java Setup (check: [0. Java Environment Setup](#)). Inside the `lw01` folder, create a subfolder named `unguided`.
2. Complete all of your work for this assessed lab inside the `unguided` folder.
3. Commit and push your work to GitHub with the commit message:
"Labwork 01 - Unguided"

To give a clearer picture, The repository for this semester's practicum will follow the structure below:

```
dsa-[nrp]/
├──
└── src/
    ├── lw01/
    │   ├── prelab/
    │   │   └── [your prelab files]
    │   └── unguided/
    │       └── [your unguided files]
    ├── lw02/
    │   ├── prelab/
    │   │   └── [your prelab files]
    │   └── unguided/
    │       └── [your unguided files]
    ├── lw03/
    │   ├── prelab/
    │   │   └── [your prelab files]
    │   └── unguided/
    │       └── [your unguided files]
    └── ...
```